📚 DSA Practical Assignment: Library Management System

Course: Data Structures & Algorithms (DSA)

Target Audience: Computer Science & Engineering Students

📖 Real-World Scenario

You are hired by your college library to build a **Smart Library Management System**.

The library has hundreds of books arranged on shelves. Students use a computer terminal to:

1. Clean up duplicate book records scanned by barcode readers.
2. Search for books by typing part of the book title (e.g., typing `"java"` to find `"Java Programming"`).
3. Sort books by price so students can see affordable books first.
4. Quickly search if a book with an exact price exists using fast searching.
5. Help students find the **minimum number of consecutive books** on a shelf whose combined value meets or exceeds a target research grant amount S.

You need to write a Java program containing **5 simple methods**, one for each task.

📋 Task Breakdown

Task #	Description	Method Signature
Task 1	Store books & remove duplicate records in-place	`int removeDuplicates(Book[] books, int n)`
Task 2	Search books using partial title (case-insensitive)	`void searchByTitle(Book[] books, int count, String query)`
Task 3	Sort books by price in ascending order	`void sortByPrice(Book[] books, int count)`
Task 4	Search a book by price in $O(\log N)$ time	`int searchByPrice(Book[] books, int count, double targetPrice)`
Task 5	Find minimum consecutive books whose total price >= S	`int minBooksForTargetCost(Book[] books, int count, double targetCost)`

🛠️ Data Model

Create a simple `Book` class:

```
class Book {
    int bookId;
    String title;
    String author;
    double price;

    public Book(int bookId, String title, String author, double price) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.price = price;
    }
}
```

📝 Detailed Task Instructions

Task 1: Remove Duplicate Books In-Place

- **Problem**: When books are scanned into the system, the scanner sometimes scans the same book twice. Given an array of books sorted by `bookId`, remove duplicates **in-place** so that each book appears only once.
- **Requirements**:
- Do **NOT** create a new array (use $O(1)$ extra memory).
- Return the new count of unique books M.
- **Method Signature**:

```
public static int removeDuplicates(Book[] books, int n)
```

Task 2: Partial Title Search

- **Problem**: A student cannot remember the full title of a book. They only type a word or part of the title (e.g., `"data"`).
- **Requirements**:

- Search through the array and find all books whose titles contain the search word.
- The search must be **case-insensitive** (e.g., `"java"` should match `"Java Programming"` and `"ADVANCED JAVA"`).
- Print all matching books.
- **Method Signature**:

```
public static void searchByTitle(Book[] books, int count, String query)
```

Task 3: Sort by Price

- **Problem**: Sort the unique books in **ascending order of price** (cheapest book first).
- **Requirements**:
- Implement standard Sort (find the minimum element in the unsorted part and swap it with the first unsorted position).
- Count and print how many swaps were performed.
- **Method Signature**:

```
public static void sortByPrice(Book[] books, int count)
```

Task 4: Search by Price

- **Problem**: Given the books array (which is now sorted by price from Task 3), check if there is a book with an exact price `targetPrice`.
- **Requirements**:
- Implement find the book in $O(\log N)$ time.
- If found, return the index of the book.
- If not found, return `-1`.
- **Method Signature**:

```
public static int searchByPrice(Book[] books, int count, double targetPrice)
```

Task 5: Problem Solving Approach — Sliding Window

- **Problem**: A research student needs to borrow a sequence of consecutive books from a library shelf. The student has a research grant requiring the total value of borrowed consecutive books to be at least **Rs.** S.
- **Requirements**:
- Find the **minimum length of a contiguous subarray of books** whose total price is >= S.

- Expand the window by adding books from the right (`right` pointer).
- Whenever `currentSum >= targetCost`, record the window length and contract the window from the left (`left` pointer) to find the smallest valid length.
- If no such contiguous group of books exists whose sum is >= S, return `0`.
- The solution must run in $O(N)$ **time** and $O(1)$ **auxiliary space**.
- **Method Signature**:

```
public static int minBooksForTargetCost(Book[] books, int count, double targetCost)
```

🧪 Sample Test Case for Students

Initial Input Array (6 Books with 1 Duplicate):

```
Book 1: ID = 101, Title = "Data Structures", Author = "Mark",   Price = 400.0
Book 2: ID = 101, Title = "Data Structures", Author = "Mark",   Price = 400.0  <-- (Duplicate)
Book 3: ID = 102, Title = "Java Basics",     Author = "James",  Price = 300.0
Book 4: ID = 103, Title = "Python Guide",    Author = "Guido",  Price = 600.0
Book 5: ID = 104, Title = "Database Systems",Author = "Raghu",  Price = 500.0
Book 6: ID = 105, Title = "Computer Networks",Author = "Andrew",Price = 700.0
```

Expected Output Step-by-Step:

1. After Task 1 (Remove Duplicates):

```
Unique Books Count: 5
Book List:
[101] Data Structures - Rs. 400.0
[102] Java Basics - Rs. 300.0
[103] Python Guide - Rs. 600.0
[104] Database Systems - Rs. 500.0
[105] Computer Networks - Rs. 700.0
```

2. After Task 2 (Search Query: "data"):

```
Search Results for 'data':
- Found: [101] Data Structures (Rs. 400.0)
- Found: [104] Database Systems (Rs. 500.0)
```

3. After Task 3 (Sort by Price):

```
Books Sorted by Price:
1. [102] Java Basics - Rs. 300.0
2. [101] Data Structures - Rs. 400.0
3. [104] Database Systems - Rs. 500.0
4. [103] Python Guide - Rs. 600.0
5. [105] Computer Networks - Rs. 700.0
Total Swaps: 2
```

4. After Task 4 (Search for Price: 500.0):

```
Searching for Price Rs. 500.0...
Result: Book found at index 2: [104] Database Systems (Rs. 500.0)
```

5. After Task 5 (Sliding Window for Target Cost S = Rs. 1000.0):

```
Finding minimum consecutive books whose total price >= Rs. 1000.0...
Minimum Consecutive Books Needed: 2

Explanation:
- Window 1 (Index 2 to 3):
  [104] Database Systems (Rs. 500.0) + [103] Python Guide (Rs. 600.0)
  Total = Rs. 1100.0 (>= 1000.0) -> Length: 2 books

- Window 2 (Index 3 to 4):
  [103] Python Guide (Rs. 600.0) + [105] Computer Networks (Rs. 700.0)
  Total = Rs. 1300.0 (>= 1000.0) -> Length: 2 books
```